VERGE

APPLICATION SECURITY ASSESSMENT REPORT

API SECURITY, ACCESS CONTROL AND DATA PROTECTION ARCHITECTURE

CLIENT: OWASP Foundation
RELEASED: 09 November, 2026

alex.vergesec@gmail.com

CLASSIFIED AS CONFIDENTIAL

TABLE OF CONTENTS

APPLICATION SECURITY ASSESSMENT REPORT 3
SECTION 1: EXECUTIVE SUMMARY & ENGAGEMENT OVERVIEW 4
1.1 Executive Risk Narrative 4
1.2 Engagement Scope Boundaries 4
1.3 Consolidated Findings Inventory (Vulnerabilities) 5
1.4 Verified Security Controls Inventory (PASS Observations) 5
SECTION 2: DETAILED VULNERABILITY FINDINGS 6
Finding 2.1: Application Prone to SQL Injection 6
Finding 2.2: Permanent Account Takeover via Sensitive Credential Disclosure in API Response & Insecure Client Storage 9
Finding 2.3: Incomplete File Content Validation & Public Static Asset Exposure 12
Finding 2.4: Information Disclosure via Unprotected Confidential Files & Unhandled Stack Trace Leakage 14
Finding 2.5: Server-Side Request Forgery (SSRF) via Unvalidated Remote Resource Fetching 17
Finding 2.6: Arbitrary Author Impersonation & Review Spoofing via Untrusted JSON Parameter 19
Finding 2.7: Systemic Insecure Direct Object Reference (IDOR) & Cross-Account Cart/Avatar Tampering 21
SECTION 3: VERIFIED DEFENSIVE CONTROLS (PASS OBSERVATIONS) 24
CTRL-01: Filename Normalization, Extension Enforcement, and Path Traversal Resistance on Media Subsystem 24
CTRL-02: Restricted File Extension Whitelisting & Directory Traversal Protection on Static File Subsystem 24
CTRL-03: File Upload Size Limitation & Request Body Throttling 25
CTRL-04: Stack Execution Isolation & Client-Side Image Context Binding 25
SECTION 4: APPENDICES & FORMAL SIGN-OFF 26
Appendix A: CVSS v3.1 Severity & Remediation Priority Framework 26
Appendix B: Developer Remediation Action Plan Checklist 27
Appendix C: Assessment Environment & Testing Tooling 27
Appendix D: Artifact Cleanup Attestation & Formal Sign-Off 28

APPLICATION SECURITY ASSESSMENT REPORT

Web Application Penetration Testing & Security Architecture Review

PROPERTY	SPECIFICATION DETAIL
TARGET APPLICATION	OWASP Juice Shop
APPLICATION VERSION	v20.2.0 (Local Deployment)
TARGET ARCHITECTURE	REST API & Node.js / Angular Web Subsystem
TESTING SCOPE	REST API Subsystems, Auth, Profile, Search, and Static Asset Storage
TESTING METHODOLOGY	Black-Box / Grey-Box Web Application Penetration Testing
ACTIVE TESTING WINDOW	October 5, 2026 – October 19, 2026
REPORT COMPLETION DATE	October 21, 2026
ASSESSOR ENTITY	Verge Security Labs

Document Administration & Control

Version Control

AUTHOR	DELIVERY DATE	PAGES	VERSION	STATUS
Alex Vance	November 9, 2026	28	v1.0	Final Deliverable

Document Distribution

NAME	TITLE	ORGANIZATION
Engineering Leadership Team	Chief Technology Officer	OWASP Foundation

SECTION 1: EXECUTIVE SUMMARY & ENGAGEMENT OVERVIEW

1.1 Executive Risk Narrative

Between September {{TESTING_START_DATE}}, 2026 and September {{TESTING_END_DATE}}, 2026, an offensive security evaluation was conducted against OWASP Juice Shop (v20.2.0 local deployment), evaluating the resilience of REST API, authentication, file upload, product search, and static file repositories under a combined Black-Box and Grey-Box methodology . Testing identified 7 security vulnerabilities (1 Critical, 5 Medium, 1 Low), including unauthenticated administrative account takeover via SQL injection on login (POST /rest/user/login), inferential boolean-based database exfiltration via product search (GET /rest/products/search), and systemic authorization flaws across cart and review endpoints.

Conversely, technical testing verified several resilient defensive baseline controls (PASS observations). These include strict filename normalization and path traversal resistance on profile uploads, extension allowlisting on static file repositories (/ftp/), request stream size throttling (~500 KB Multer limit), and client-side stack execution isolation.

Real-World Business Risks

• **Unauthenticated Administrative ATO via SQL Injection:** Unauthenticated adversaries can inject boolean breakout payloads (' OR 1=1--) into POST /rest/user/login to bypass password verification and obtain administrative session tokens.
• **Inferential Database Exfiltration via Search Oracle:** Attackers can leverage response differentials in GET /rest/products/search (q parameter) to systematically exfiltrate database schemas, user records, and password hashes.
• **Cross-Account Authorization Breakdown (IDOR & Impersonation):** Flaws in object reference handling permit unauthorized viewing of victim shopping carts (basketId) and arbitrary author handle spoofing on published reviews (author JSON parameter).
• **Out-of-Band SSRF & Confidential File Exposure:** Unvalidated URL parameter fetching in /profile/image/url allows attackers to trigger outbound server calls, while unlinked file paths in /ftp/ expose confidential corporate acquisition documents.

1.2 Engagement Scope Boundaries

TARGET ENDPOINT / URL	ENVIRONMENT	TESTING SCOPE
POST /rest/user/login & GET /rest/products/search	Staging / Local v20.2.0	Primary Authentication & Product Catalog Search Audit (In-Scope)
GET /rest/basket/{basketId} & POST /rest/products/reviews	Staging / Local v20.2.0	Cart Subsystem & Review Management Access Controls (In-Scope)
POST /profile/image/file & POST /profile/image/url	Staging / Local v20.2.0	Profile Avatar Uploads & Remote Image Fetching (In-Scope)
GET /robots.txt & GET /ftp/*	Staging / Local v20.2.0	Static Document Repository & Directory Listing Protections (In-Scope)

1.3 Consolidated Findings Inventory (Vulnerabilities)

POINT	TITLE	CWE-ID	CWE CATEGORY	SEVERITY
FIND-01	Application Prone to SQL Injection	CWE-89 / CWE-287	A03:2025 – Injection	**CRITICAL**
FIND-02	Permanent Account Takeover via Sensitive Credential Disclosure in API Response & Insecure Client Storage	CWE-200 / CWE-312 / CWE-328	A04:2025 – Cryptographic Failures	**MEDIUM**
FIND-03	Incomplete File Content Validation & Public Static Asset Exposure	CWE-434 / CWE-1284	A05:2025 – Security Misconfiguration	**LOW**
FIND-04	Information Disclosure via Unprotected Confidential Files & Unhandled Stack Trace Leakage	CWE-209 / CWE-538	A05:2025 – Security Misconfiguration	**MEDIUM**
FIND-05	Server-Side Request Forgery (SSRF) via Unvalidated Remote Resource Fetching	CWE-918	A10:2025 – Server-Side Request Forgery (SSRF)	**MEDIUM**
FIND-06	Arbitrary Author Impersonation & Review Spoofing via Untrusted JSON Parameter	CWE-639 / CWE-284	A01:2025 – Broken Access Control	**MEDIUM**
FIND-07	Systemic Insecure Direct Object Reference (IDOR) & Cross-Account Cart/Avatar Tampering	CWE-639 / CWE-284	A01:2025 – Broken Access Control	**MEDIUM**

1.4 Verified Security Controls Inventory (PASS Observations)

POINT	SECURITY FEATURE VERIFIED	EVALUATED ENDPOINT	STANDARD MAPPING	STATUS
CTRL-01	Filename Normalization, Extension Enforcement & Path Traversal Resistance	POST /profile/image/file & GET /assets/public/images/uploads/{filename}	A01:2025 – Broken Access Control / A05:2025 – Security Misconfiguration	**PASS**
CTRL-02	Restricted File Extension Whitelisting & Directory Traversal Protection	GET /ftp/*	A01:2025 – Broken Access Control / A05:2025 – Security Misconfiguration	**PASS**
CTRL-03	File Upload Size Limitation & Request Body Throttling	POST /profile/image/file	A05:2025 – Security Misconfiguration	**PASS**
CTRL-04	Stack Execution Isolation & Client-Side Image Context Binding	POST /profile/image/file & GET /assets/public/images/uploads/{filename}	A03:2025 – Injection / A05:2025 – Security Misconfiguration	**PASS**

SECTION 2: DETAILED VULNERABILITY FINDINGS

Finding 2.1: Application Prone to SQL Injection

SEVERITY: CRITICAL	CWE-ID: CWE-89 / CWE-287	CVSS SCORE: 9.8
AFFECTED POINT	POST /rest/user/login (email parameter & GET /rest/products/search)	
OWASP TOP 10	A03:2025 – Injection	

Vulnerability Description & Technical Mechanics

The web application processes untrusted user input across multiple endpoints by dynamically concatenating parameters directly into backend SQL database queries without prior sanitization, escaping, or query parameterization. This architectural defect exposes two distinct SQL injection attack vectors: Authentication Bypass on Login Handler (POST /rest/user/login) and Inferential Boolean-Based Blind SQL Injection in Product Search (GET /rest/products/search). The login handler constructs dynamic database commands using the email parameter; submitting a boolean breakout string (' OR 1=1--) forces the query's WHERE clause to evaluate as TRUE for all table records, truncating password checks and returning the primary system Administrator account (ID 1) to issue a full administrative JWT token. In the product search handler, embedding the q parameter directly into a wildcard query (WHERE name LIKE '%<input>%') permits unclosed quote/parenthesis breakout ('))). While raw data is not directly reflected in responses, the handler acts as a 1-bit Boolean response oracle (HTTP 200 OK with matching products vs empty JSON array), allowing character-by-character exfiltration of database schemas, user tables, and password hashes.

Steps to Reproduce (Proof of Concept)

Vector A: Unauthenticated Administrative Authentication Bypass

1. Intercept a standard login request sent to POST /rest/user/login using Burp Suite Proxy. Replace the email JSON parameter with the SQL injection authentication bypass payload ' OR 1=1-- and leave the password parameter blank.
2. Forward the request and observe that the server responds with HTTP 200 OK containing an administrative JWT session token for admin@juice-sh.op, achieving full unauthenticated administrative account takeover.

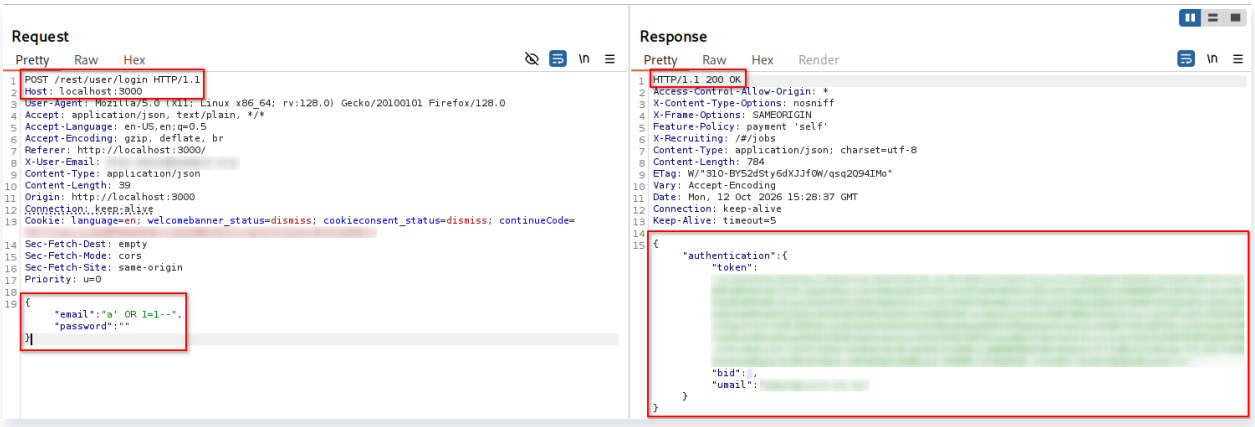


Figure 2.1A: Burp Proxy split pane showing modified POST /rest/user/login request with ' OR 1=1-- alongside successful 200 OK response containing administrative JWT token

Vector B: Character-by-Character Inferential Boolean-Based Data Exfiltration

3. To test inferential boolean-based SQL injection, navigate to GET /rest/products/search?q=apple')) AND 1=1-- in Burp Repeater. Observe that the TRUE condition evaluates successfully, returning matching catalog items in the JSON array.

4. Modify the query payload to a FALSE condition (GET /rest/products/search?q=apple')) AND 1=2--). Observe that the server returns HTTP 200 OK with an empty JSON array ([]), establishing a 1-bit Boolean response oracle for SQLite database version exfiltration via Burp Intruder.

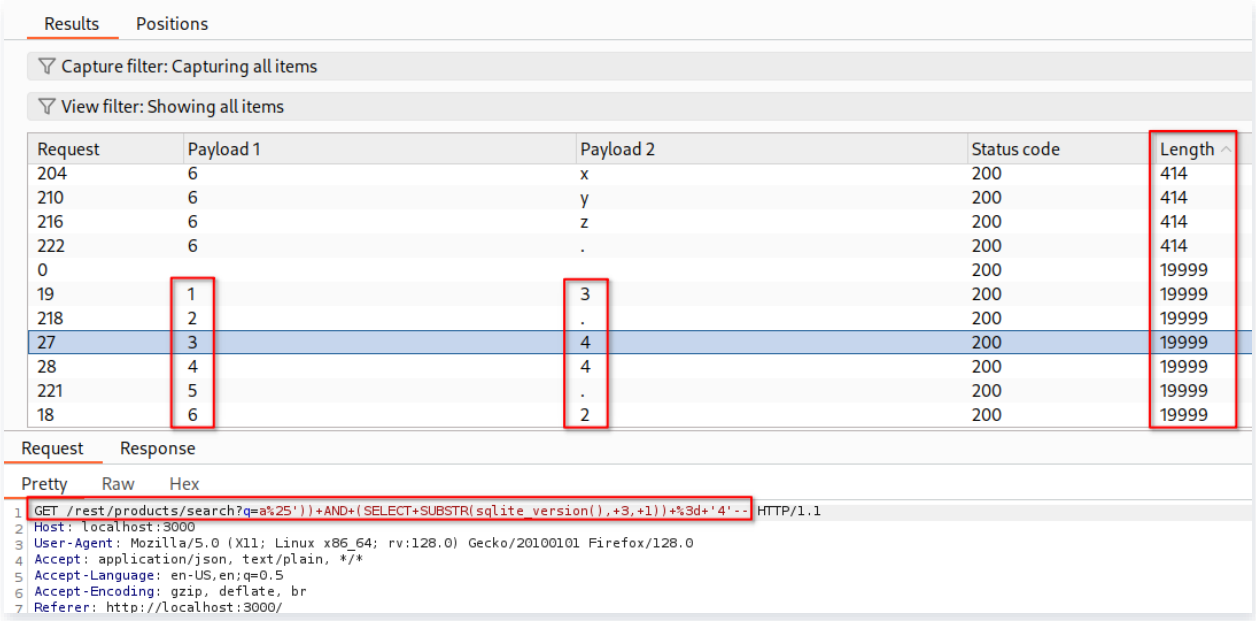


Figure 2.1B: Burp Intruder interface displaying the Boolean differential between TRUE and FALSE search query conditions

Risk & Business Impact

Exploiting these SQL injection vulnerabilities permits unauthenticated threat actors to achieve total application compromise. An attacker can instantly hijack the primary administrator account without credentials, bypass authentication mechanisms, and systematically extract the entire backend database (including user credentials, session tokens, and personal identifiable information). Inferential blind data extraction circumvents standard output-filtering web application firewalls (WAFs), posing a critical threat to corporate confidentiality, data integrity, and regulatory compliance.

Remediation Guidance

1. **Enforce Parameterized SQL Queries & Prepared Statements:** Re-architect all database query handlers across POST /rest/user/login and GET /rest/products/search to use parameterized queries (prepared statements) via ORM models (e.g., Sequelize) or parameterized database bindings (db.all('SELECT * FROM Users WHERE email = ?', [email])). This separates code logic from user-supplied data, neutralizing SQL injection.
2. **Implement Strict Server-Side Input Validation & Encoding:** Validate all incoming parameter strings against strict allowlists before constructing database queries. Reject unexpected metacharacters or SQL control characters (', ", --, ;) at the API validation layer.
3. **Enforce Principle of Least Privilege for Database Accounts:** Restrict the backend database execution service account to minimal required table read/write permissions, revoking administrative database procedures and file system access.

Finding 2.2: Permanent Account Takeover via Sensitive Credential Disclosure in API Response & Insecure Client Storage

SEVERITY: MEDIUM	CWE-ID: CWE-200 / CWE-312 / CWE-328	CVSS SCORE: 5.3
AFFECTED POINT	POST /rest/user/change-password & Client Session Storage (localStorage / Cookies)	
OWASP TOP 10	A04:2025 – Cryptographic Failures	

Vulnerability Description & Technical Mechanics

The application exhibits severe cryptographic and serialization flaws during credential management and session handling. When a user executes a password change request at POST /rest/user/change-password, the backend API over-serializes the response payload, returning the complete updated user database object—including the user's unsalted MD5 password hash (password property). Furthermore, the client-side single-page application stores session JWT tokens inside unencrypted browser localStorage without HttpOnly flags. Decoding (Base64) the JWT payload reveals that identical user credential attributes are mirrored directly within client-side memory. Because MD5 is cryptographically broken and lacks computational hardness, adversaries can extract these hashes from network proxy logs, server access logs, or XSS-harvested localStorage tokens and execute offline GPU brute-force attacks to recover plaintext credentials, leading to permanent Account Takeover (ATO).

Steps to Reproduce (Proof of Concept)

Vector A: API Response Credential Disclosure

1. Authenticate as a standard user account and issue a password update request to POST /rest/user/change-password.
2. Inspect the returned HTTP 200 OK response body in Burp Repeater. Observe that the backend over-serializes the user record, disclosing the user's updated MD5 password hash in plain text inside the JSON response payload.

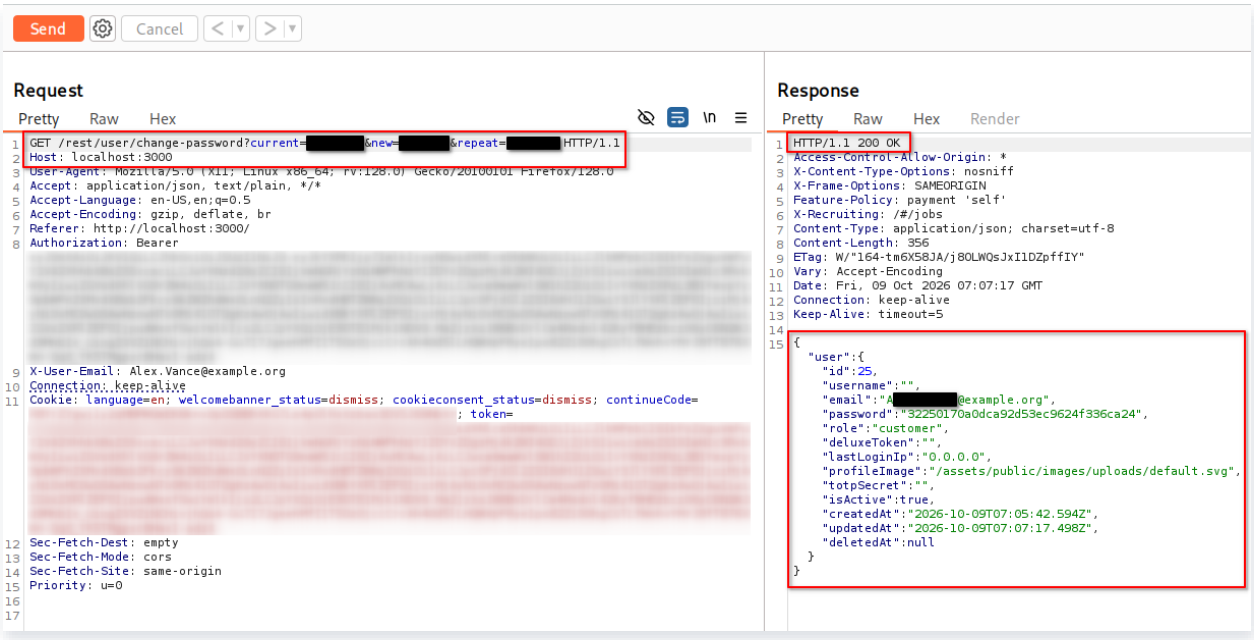


Figure 2.2A: Burp Repeater split pane displaying API response metadata revealing clear text user details, session claims, and password hashes within the returned JSON

Vector B: Insecure Client-Side Storage

3. Inspect browser client storage (localStorage / Session Cookies) via Developer Tools. Confirm that session JWT tokens are stored without HttpOnly protection, exposing Base64-encoded token payloads containing user identity attributes accessible to client-side scripts.

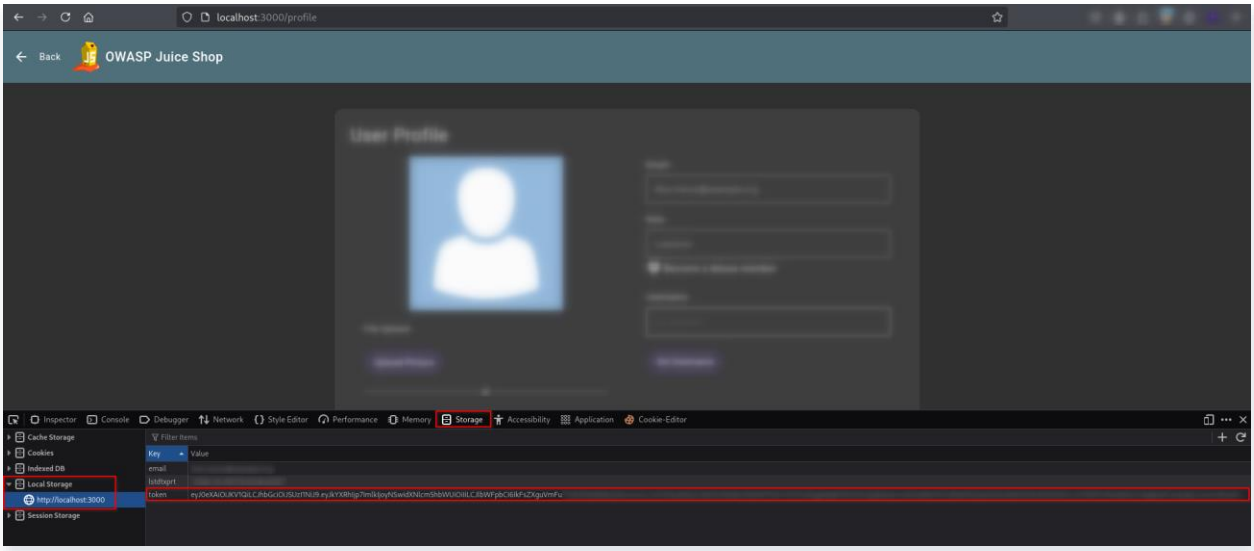


Figure 2.2B: Unencrypted JWT session token exposed in browser Local Storage

Risk & Business Impact

Over-serializing sensitive database fields exposes unsalted password hashes within unencrypted web server logs, API proxies, and client memory. Combined with MD5's weakness against offline GPU cracking dictionary attacks, threat actors can convert temporary session access or log exposure into permanent account takeover.

Remediation Guidance

1. **Eliminate Credential Over-Serialization in API Responses:** Re-architect user object serialization handlers to explicitly exclude sensitive password hash fields (password, passwordHash, salt) from JSON response schemas. Return only essential status indicators.
2. **Migrate to Strong Password Hashing Algorithms (Argon2id / bcrypt):** Replace legacy unsalted MD5 hashing routines with modern, computationally hard password hashing algorithms such as Argon2id or bcrypt (work factor >= 12) with unique per-user salts.
3. **Store Session Tokens in Secure HttpOnly Cookies:** Move session tokens out of client-accessible localStorage into server-managed, HTTP-only, secure cookies (Set-Cookie: token=...; Secure; HttpOnly; SameSite=Strict) to prevent script-based token harvesting.

Finding 2.3: Incomplete File Content Validation & Public Static Asset Exposure

SEVERITY: LOW	CWE-ID: CWE-434 / CWE-1284	CVSS SCORE: 3.1
AFFECTED POINT	POST /profile/image/file (filename parameter)	
OWASP TOP 10	A05:2025 – Security Misconfiguration	

Vulnerability Description & Technical Mechanics

The user profile avatar upload subsystem validates file uploads solely by checking binary magic bytes in the file header. This superficial validation can be easily bypassed using metadata tools (such as exiftool) to embed PNG magic bytes alongside executable script tags inside image EXIF metadata chunks, allowing polyglot files (payload.php) to pass content validation. While direct Remote Code Execution (RCE) is currently mitigated because the backend forcibly renames uploaded files to static .png extensions (e.g., 26.png) and serves static directories without server-side script execution engines (such as PHP-FPM), uploaded assets are stored directly within a public webroot path (assets/public/images/upload/26.png) accessible to unauthenticated users without session tokens or access controls.

Steps to Reproduce (Proof of Concept)

1. Construct a polyglot file (test_payload.php) containing valid PNG magic bytes in the header and embedded script payloads inside metadata chunks using exiftool. Intercept the avatar upload request to POST /profile/image/file in Burp Repeater.



Figure 2.3A: File upload validation bypass request on POST /profile/image/upload

2. Observe that the server accepts the upload with HTTP 200 OK, bypassing magic byte validation and assigning the static filename 26.png.

3. Strip all authorization headers and session cookies, then issue a direct unauthenticated GET request to http://localhost:3000/assets/public/images/uploads/26.png. Observe that the server returns HTTP 200 OK, publicly serving the uploaded polyglot image asset without requiring authentication.

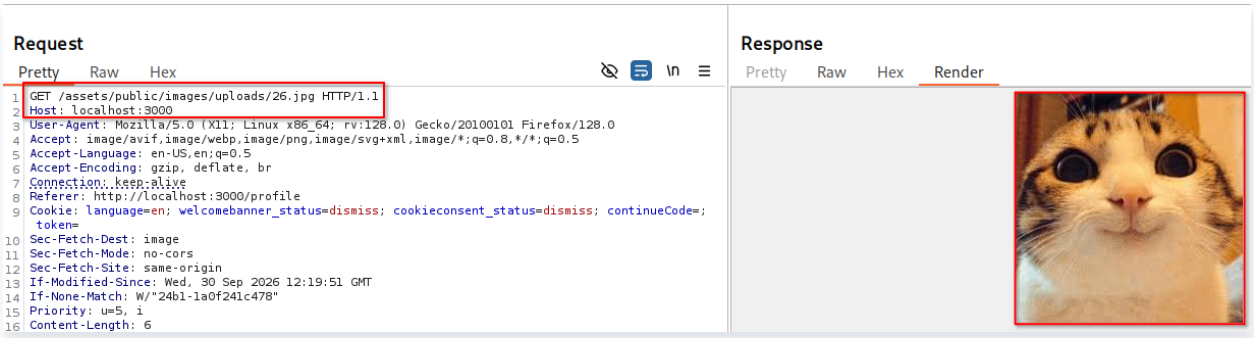


Figure 2.3B: Direct unauthenticated browser access to uploaded asset in /uploads/avatars

Risk & Business Impact

Relying on magic byte validation permits attackers to store unsanitized polyglot payloads on disk. Public static asset exposure allows unauthenticated media harvesting. Furthermore, relying on extension renaming as the sole execution barrier creates latent RCE risk if server routing or script handler configurations change in future deployment cycles.

Remediation Guidance

1. Server-Side Image Re-Encoding & EXIF Stripping: Process all uploaded image files through server-side image processing libraries (e.g., Sharp, ImageMagick) to re-encode image pixels and strip all non-image metadata chunks, EXIF tags, and embedded code bytes before writing files to disk.

2. Isolate Static Upload Storage Outside Webroot: Move user upload storage outside the public web server execution root or store assets in an isolated cloud storage bucket (e.g., AWS S3) with script execution strictly disabled.

3. Enforce Access Controls on Uploaded Assets: Restrict direct static file access by serving user media through an authenticated API controller that validates session permissions before streaming asset files.

4. Enforce Strict Server-Side Extension Allowlisting: Restrict uploaded files exclusively to a strict allowlist of approved image extensions (e.g., .jpg, .png, .webp) at the perimeter level, rejecting unapproved or dangerous extensions prior to disk storage.

Finding 2.4: Information Disclosure via Unprotected Confidential Files & Unhandled Stack Trace Leakage

SEVERITY: MEDIUM	CWE-ID: CWE-209 / CWE-538	CVSS SCORE: 5.3
AFFECTED POINT	GET /robots.txt (/ftp/* & POST /rest/user/login)	
OWASP TOP 10	A05:2025 – Security Misconfiguration	

Vulnerability Description & Technical Mechanics

The application exhibits multiple information disclosure vulnerabilities that expose sensitive corporate intelligence and backend runtime architecture to unauthenticated threat actors across two distinct attack vectors: Unprotected Corporate Artifact Exposure (robots.txt & /ftp/ Directory) and Unhandled Server-Side Exception & Stack Trace Leakage (POST /rest/user/login). Under Vector 1, the web application publishes a public robots.txt file referencing the /ftp/ directory; because directory listing and file download restrictions are missing, unauthenticated external actors can directly download sensitive internal documents (/ftp/acquisitions.md) containing confidential strategic business disclosures, planned competitor acquisitions, and shareholder briefings labeled with explicit confidentiality warnings. Under Vector 2, submitting a JSON array structure into the login handler password field causes crypto.Hash.update() to throw an unhandled TypeError, prompting the application to return an HTTP 500 response leaking a raw Node.js/Express stack trace and absolute backend filesystem paths.

Steps to Reproduce (Proof of Concept)

Vector A: Unprotected Confidential File Exposure

1. Open http://localhost:3000/robots.txt in a web browser, identify the disallowed /ftp/ directive, and issue an unauthenticated GET request to http://localhost:3000/ftp/acquisitions.md.

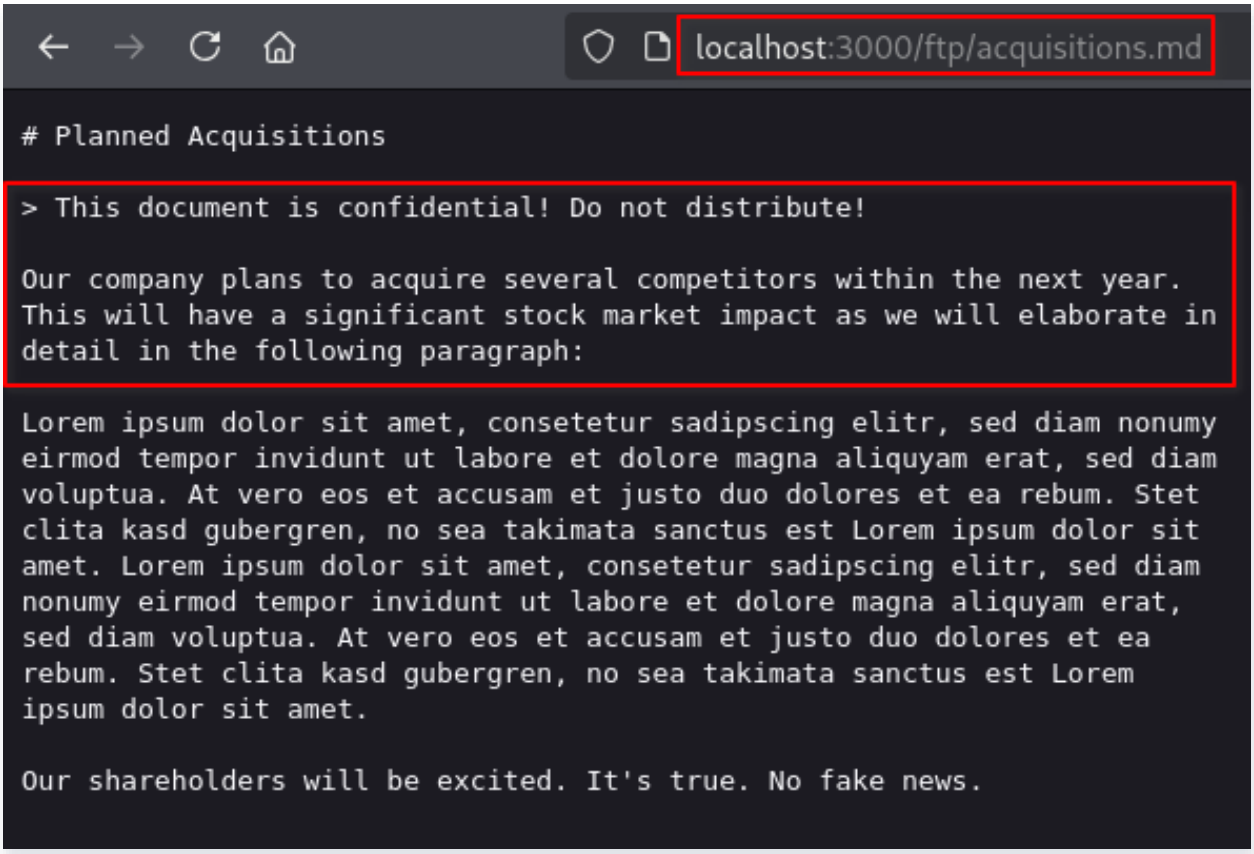


Figure 2.4A: Unauthenticated file download of exposed confidential backups via the /ftp/ directory

2. Observe that the server returns HTTP 200 OK containing sensitive corporate M&A strategy communications and explicit insider market disclosures.

Vector B: Unhandled Stack Trace Leakage via Array Parameter Injection

3. Intercept a login request targeting POST /rest/user/login in Burp Repeater. Modify the password parameter into a JSON array structure.

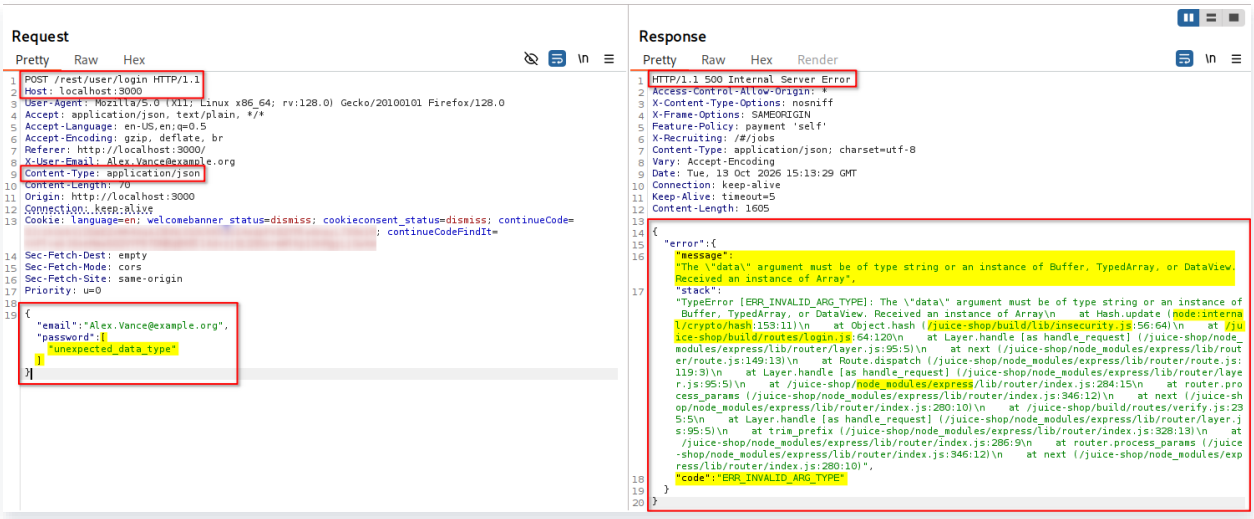


Figure 2.4B: Unhandled Express.js stack trace leakage triggered via JSON array injection on POST /rest/user/login

4. Submit the request and observe the server return HTTP 500 Internal Server Error with a raw Node.js stack trace exposing local backend directory paths and module dependencies.

Risk & Business Impact

Direct public availability of confidential corporate communications exposes internal strategic initiatives and M&A activities, bringing reputational damage and regulatory non-compliance. Concurrently, stack trace disclosures provide attackers with internal filesystem layouts and technology stack maps to refine targeted exploits.

Remediation Guidance

1. Relocate Confidential Files & Restrict Directory Access: Remove confidential corporate documents from publicly accessible web server roots (/public, /ftp). Enforce strict authentication and role-based access control (RBAC) middleware on all file serving routes.

2. Implement Global Graceful Exception Handling: Configure centralized production error-handling middleware in Express.js. Intercept all unhandled exceptions globally and return generic error messages (HTTP 500 Internal Server Error) without disclosing internal stack traces or filesystem paths.

3. Enforce Strict API Schema & Data Type Validation: Validate incoming API payload data types against strict JSON schemas (e.g., using Ajv or Joi) before passing parameters to business logic routines. Reject mismatched types with HTTP 400 Bad Request.

Finding 2.5: Server-Side Request Forgery (SSRF) via Unvalidated Remote Resource Fetching

SEVERITY: MEDIUM	CWE-ID: CWE-918	CVSS SCORE: 5.3
AFFECTED POINT	POST /profile/image/url (imageUrl parameter)	
OWASP TOP 10	A10:2025 – Server-Side Request Forgery (SSRF)	

Vulnerability Description & Technical Mechanics

The application exposes a remote profile image fetching endpoint (POST /profile/image/url) that accepts user-supplied destination URLs via the imageUrl JSON parameter without enforcing IP address restrictions, domain allowlists, or protocol scheme validation. Technical evaluation confirmed that the backend Node.js application server acts as an HTTP client, issuing arbitrary outbound network requests upon receiving untrusted user input.
Out-of-band (OOB) testing utilizing an Interactsh listener verified that the endpoint is vulnerable to Server-Side Request Forgery (SSRF). Supplying a unique Interactsh URL (http://<id>.oast.fun) in the imageUrl parameter forced the backend application server to process the request and make an outbound HTTP GET call to the listener. Fuzzing internal targets (e.g., AWS IMDS at 169.254.169.254) in Burp Intruder revealed distinct response timing variance (200ms, 800ms, and 4,000–6,000ms), but could not conclusively distinguish between closed ports, WAF drops, or delayed backend processing. Consequently, speculative internal port scanning results were excluded, though the underlying SSRF primitive remains fully verified via out-of-band interaction.

Steps to Reproduce (Proof of Concept)

1. Intercept a profile picture URL submission request targeting POST /profile/image/url in Burp Suite Proxy and transfer the transaction to Burp Repeater. Generate a unique listener URL using Interactsh (e.g., http://<id>.oast.fun/test) and inject it into the imageUrl JSON request body parameter:

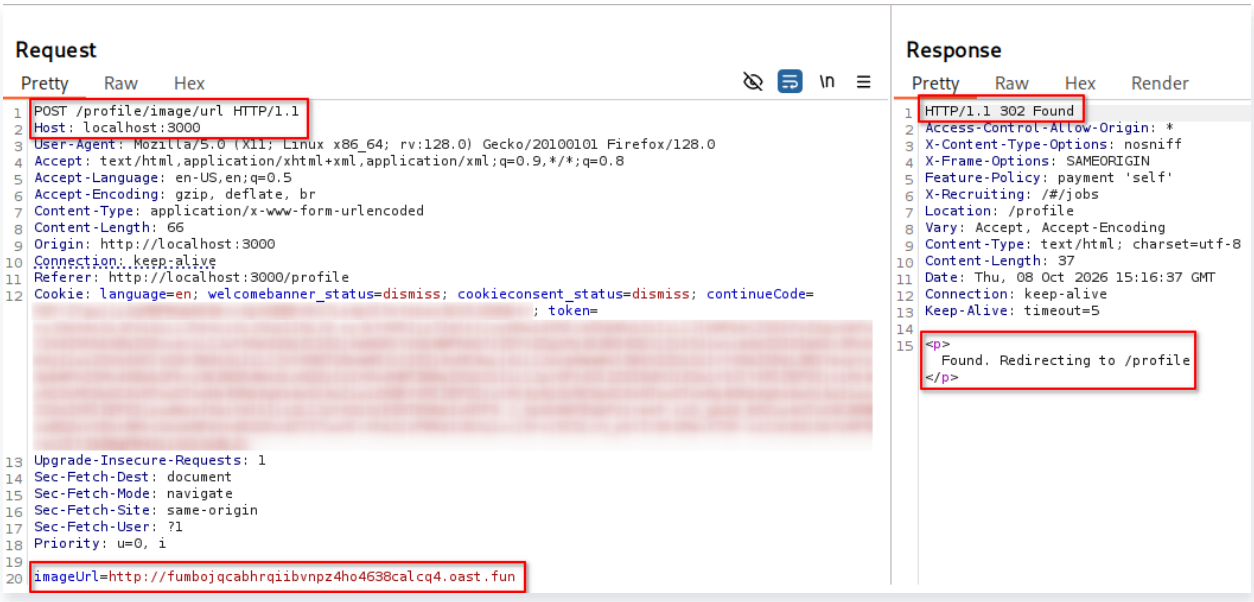


Figure 2.5A: Burp Repeater POST /profile/image/url request injecting an Interactsh listener URL into the imageUrl parameter

2. Forward the request and inspect the Interactsh client console. Confirm that an inbound HTTP GET request is received, displaying the backend application server's runtime User-Agent header and confirming server-side execution.

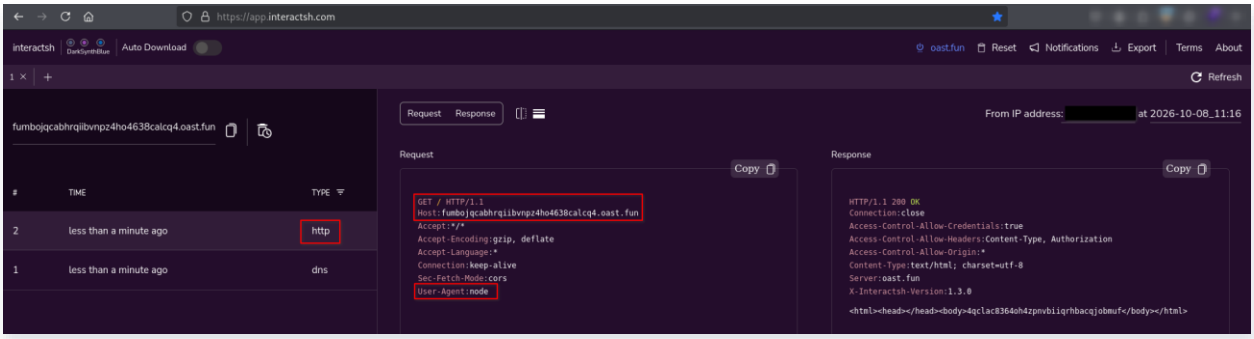


Figure 2.5B: Interactsh listener logs demonstrating confirmed out-of-band HTTP interaction from the backend server

Note: Sending requests via Burp Suite won't trigger Interactsh logs unless the backend server executes the outbound call itself (or unless the request was made via browser). Because the application was tested locally (localhost), the server's outbound request egressed through the host machine's IP, matching the tester's IP. The received HTTP callback on Interactsh confirms successful server-side execution.

Risk & Business Impact

Exploiting this Server-Side Request Forgery vulnerability permits unauthenticated attackers to abuse the application server as an open egress proxy. Threat actors can leverage this primitive to blindly spray payloads across internal subnets, loopback services, and metadata endpoints—banking on hitting unauthenticated microservices or management interfaces without needing prior visibility into the internal network topology.

Remediation Guidance

1. **Enforce Strict IP & Domain Destination Allowlists:** Validate user-supplied destination URLs against a strict server-side domain allowlist before issuing outbound HTTP requests. Reject any requests targeting unapproved domains or IP addresses.
2. **Block Loopback, RFC 1918 Private Subnets & Cloud IMDS:** Resolve target hostnames server-side prior to execution and explicitly reject connections targeting loopback addresses (127.0.0.1, localhost), private RFC 1918 subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), and Link-Local Cloud Metadata IPs (169.254.169.254).
3. **Implement Egress Network Segmentation & Firewall Restrictions:** Configure network security groups and firewall egress rules to restrict the application server container from initiating arbitrary TCP connections to internal subnets or cloud management endpoints.

Finding 2.6: Arbitrary Author Impersonation & Review Spoofing via Untrusted JSON Parameter

SEVERITY: MEDIUM	CWE-ID: CWE-639 / CWE-284	CVSS SCORE: 5.3
AFFECTED POINT	PUT /rest/products/1/reviews (author JSON parameter)	
OWASP TOP 10	A01:2025 – Broken Access Control	

Vulnerability Description & Technical Mechanics

The product feedback and review subsystem allows authenticated users to post ratings and public comments on catalog items. However, the backend REST API fails to validate or enforce session identity boundaries when processing review submissions. When a user submits a review, the web application transmits a JSON payload containing the product ID, rating, review message, and an explicit author string property specifying the submitter's identity. Rather than deriving the submitter's identity from verified server-side session claims (such as the decoded email or sub claim inside the Authorization: Bearer JWT), the API endpoint blindly trusts the client-controlled author parameter. An adversary can construct or intercept a review submission request and replace their own email address with any target handle—including administrative accounts such as admin@juice-sh.op. The server processes the request without cross-checking the active session token or requiring victim credentials, publishing the comment publicly under the victim's account.

Steps to Reproduce (Proof of Concept)

1. Authenticate as a standard user account, navigate to a product detail view, and intercept the product review submission sent to PUT /rest/products/1/reviews in Burp Suite Repeater. Observe the client-controlled author parameter in the JSON payload.

2. Modify the author parameter value from the authenticated user's email to the target administrator account (admin@juice-sh.op), or any other victims account that you might be able to find under product review. Submit the request and observe the server respond with HTTP 201 Created.

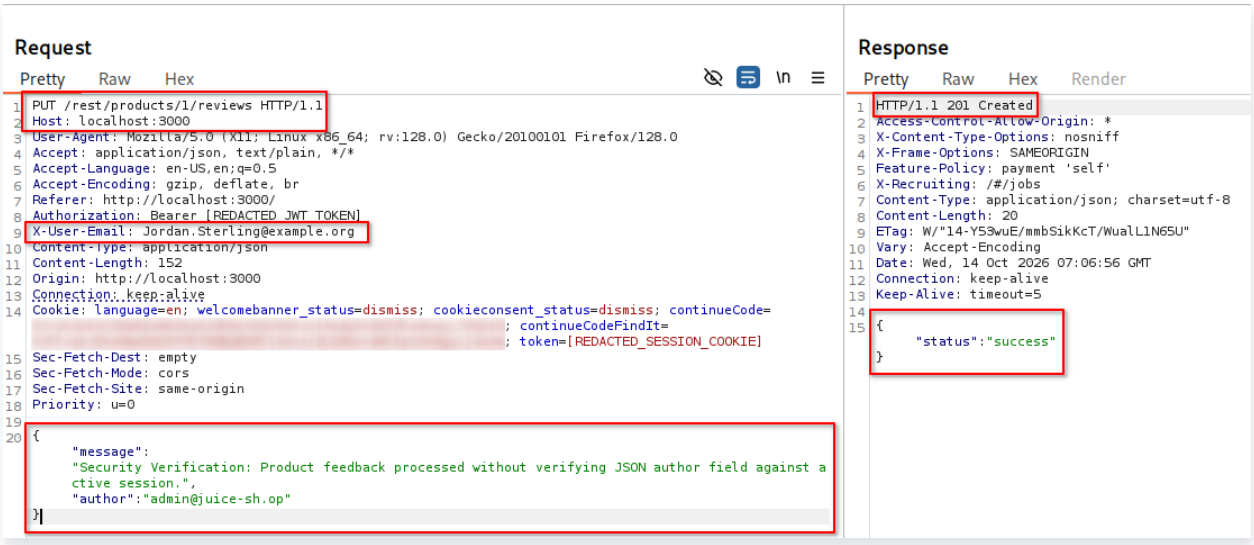


Figure 2.6A: Burp Repeater tampering author to admin@juice-sh.op returning "201 Created"

3. Refresh the product detail modal in the web browser. Confirm that the review comment is rendered publicly in the feedback section, falsely attributed to admin@juice-sh.op.

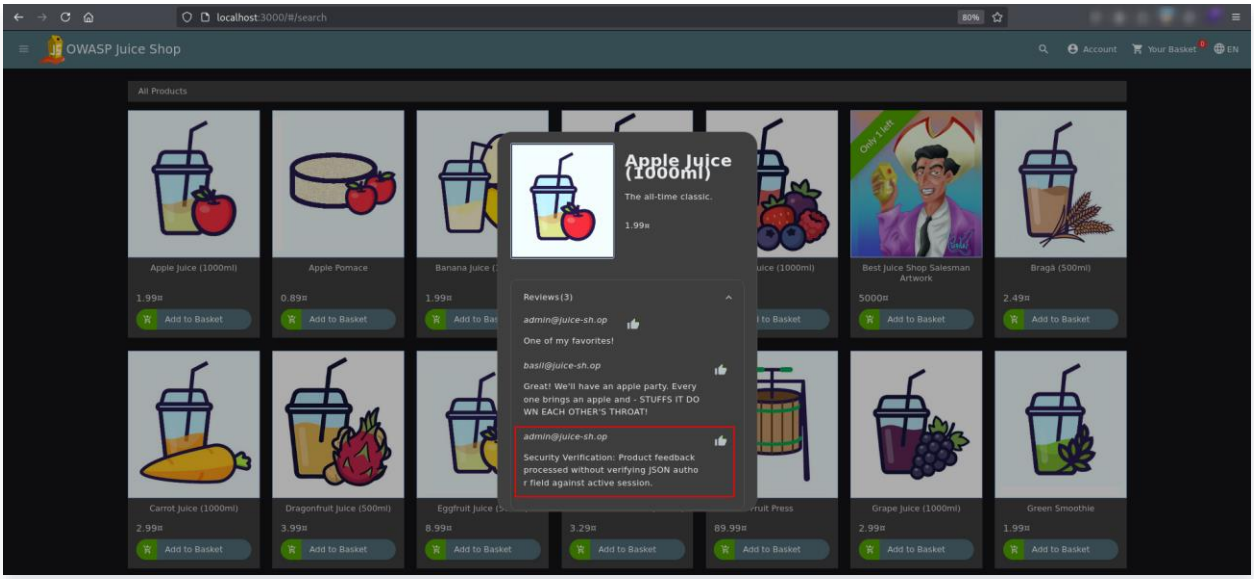


Figure 2.6B: Product review UI displaying the published comment publicly rendered under the spoofed admin@juice-sh.op handle

Risk & Business Impact

Attackers can publish arbitrary, misleading, or authoritative statements under high-privileged handles (admin@juice-sh.op) or executive emails without possessing their credentials or session tokens. This enables reputational sabotage, fake review manufacturing, and destruction of platform non-repudiation.

Remediation Guidance

1. **Bind Author Identity Strictly to Verified Session Tokens:** Remove client-controllable identity properties (author, email, userId) from incoming API request bodies. Re-architect review creation handlers to extract submitter identity exclusively from verified server-side session contexts (e.g., parsing claims from the validated JWT attached in Authorization: Bearer).
2. **Enforce Server-Side Object-Level Access Control:** If explicit user identification parameters must be accepted by an endpoint for legacy compatibility, enforce strict server-side access control checks verifying that the authenticated session identity matches the specified author string before executing database mutations.
3. **Audit Subsystem Non-Repudiation Controls:** Review all user-generated content endpoints across the REST API to guarantee that resource creation and modification actions are cryptographically and logically bound to the caller's active session.

Finding 2.7: Systemic Insecure Direct Object Reference (IDOR) & Cross-Account Cart/Avatar Tampering

SEVERITY: MEDIUM	CWE-ID: CWE-639 / CWE-284	CVSS SCORE: 5.4
AFFECTED POINT	GET /rest/basket/{basketId} & GET /assets/public/images/uploads/{id}	
OWASP TOP 10	A01:2025 – Broken Access Control	

Vulnerability Description & Technical Mechanics

The application exhibits systemic Insecure Direct Object Reference (IDOR) / Broken Object-Level Authorization (BOLA) vulnerabilities across multiple user resource endpoints (/rest/basket/{basketId} and /assets/public/images/uploads/{id}). The backend REST API utilizes sequential numeric identifiers to query user shopping carts and profile avatar images. However, the server fails to perform object-level authorization checks to verify whether the requested resource ID matches the authenticated session caller. When user J.S intercepts their outgoing shopping basket request (GET /rest/basket/7) or avatar request (GET /assets/public/images/uploads/14) and tampers with the path parameter integer—substituting 7 with victim user A.V's basket ID 6 or avatar ID 14 with 26—the backend server processes the request and returns the victim's data. Furthermore, because the rendered shopping cart GUI provides interactive controls (+ / - quantity buttons and item deletion icons), the attacker can actively modify or destroy the victim's shopping cart state.

Steps to Reproduce (Proof of Concept)

Vector 1: Horizontal Access Control Bypass & User Avatar Asset Disclosure

1. Authenticate as victim account A.V (AlexVance@example.com), upload a custom avatar (saved on the server as 26.png), and intercept the HTTP request fetching the image (GET /assets/public/images/uploads/26). Transfer the transaction to Burp Repeater and submit to observe HTTP 200 OK returning A.V's avatar under A.V's active session.



Figure 2.7A: Burp Repeater baseline request displaying victim A.V's session headers returning avatar image 26.png

2. Authenticate as attacker account J.S (JordanSterling@example.com), copy J.S's active JWT authorization token, and replace A.V's token in Burp Repeater. Submit the request targeting 26.png using J.S's JWT token and observe an HTTP 200 OK response returning A.V's avatar image without server-side ownership verification.



Figure 2.7B: Authorization bypass in Burp Repeater showing attacker J.S's JWT token successfully fetching victim A.V's avatar 26.png

Vector 2: Cross-Account Shopping Basket Inspection & Cart State Tampering

3. Authenticate as test account J.S (JordanSterling@example.com), whose shopping basket currently contains 1 product. Intercept an initial basket request targeting GET /rest/basket/7 and send it to Burp Repeater. Modify the basket ID path parameter from 7 to 6 (belonging to victim account A.V) and submit the request. Observe an HTTP 200 OK response returning A.V's sensitive basket details without secondary authorization.

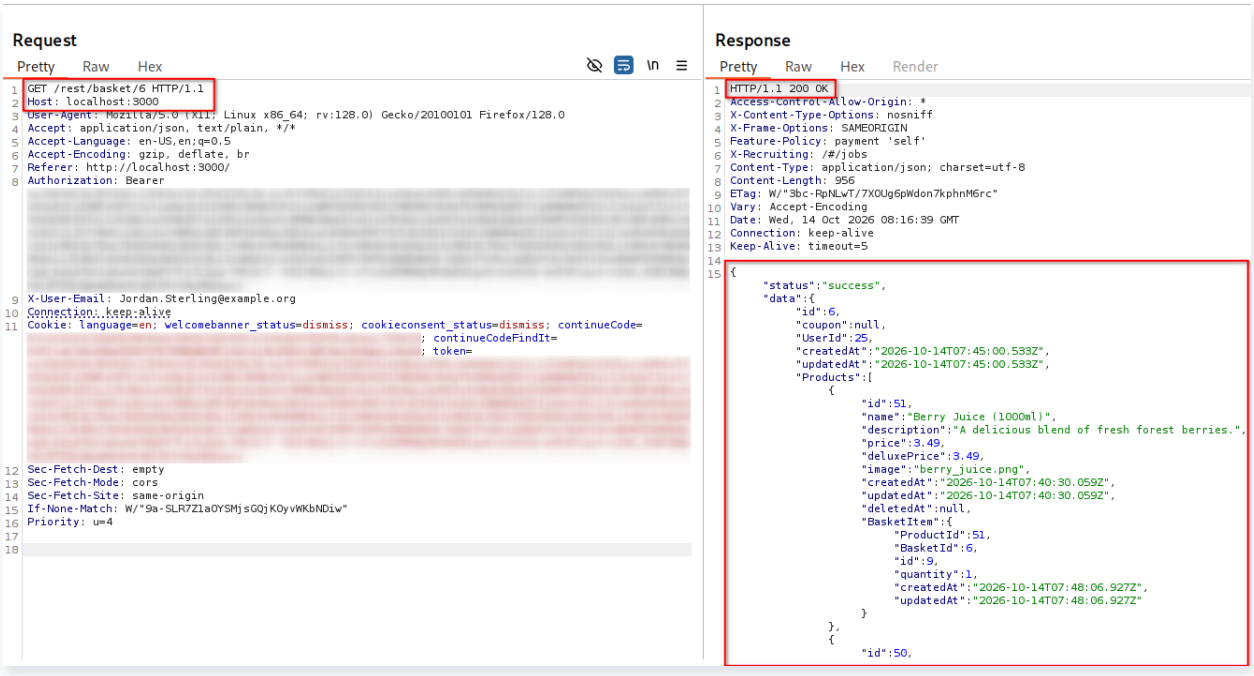


Figure 2.7C: Burp Repeater split pane displaying GET /rest/basket/6 returning HTTP 200 OK with victim A.V's basket items

4. In the web application, click 'Your Basket' (/#/basket) with Burp Proxy intercept active (Intercept is on). When the outgoing GET /rest/basket/7 request hits the Intercept tab, change path parameter 7 to 6 and click “Forward all”. Inspect the browser to confirm that while the top cart badge displays 1 (reflecting J.S's actual server-linked count), the main cart table renders A.V's 2 products, confirming full UI desynchronization and cross-account IDOR exposure.

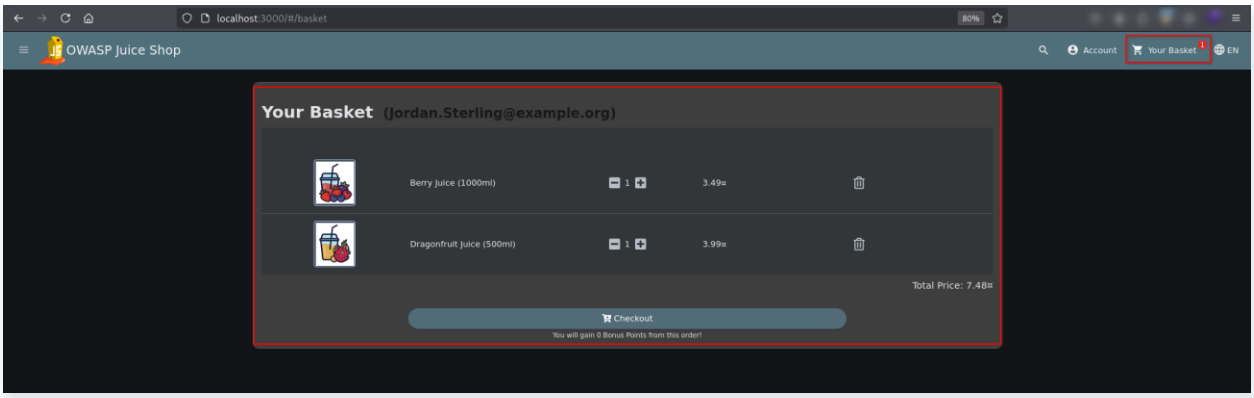


Figure 2.7D: Browser interface displaying cart badge count of 1 alongside shopping cart table rendering victim A.V's 2 products

Risk & Business Impact

Attackers can enumerate sequential integer IDs to harvest real-time product interests, order habits, profile pictures, and personal data across all registered platform users. Furthermore, attackers can delete items or alter quantities in victim carts, leading to inventory sabotage and lost revenue.

Remediation Guidance

1. Enforce Server-Side Object-Level Authorization (BOLA Mitigation): Modify backend handlers for /rest/basket/{basketId} and /assets/public/images/uploads/{id} to extract the authenticated user ID directly from the validated session token (e.g., JWT claims). Reject any request where the requested resource ID does not belong to the authenticated caller with an HTTP 403 Forbidden status code.

2. Implement Cryptographic or Non-Sequential Resource Identifiers: Replace simple auto-incrementing integer identifiers (1, 2, 3) with cryptographically secure Universally Unique Identifiers (UUIDv4) for resource objects to prevent trivial sequential ID enumeration.

3. Ensure Client-Server State Consistency: Ensure client-side UI components validate and re-synchronize item badge counts against verified backend response data to prevent authorization desynchronization.

SECTION 3: VERIFIED DEFENSIVE CONTROLS (PASS OBSERVATIONS)

As part of our holistic testing methodology, offensive security engagements evaluate not only exploitable vulnerabilities but also active defensive mechanisms that successfully mitigate potential attack vectors. The controls below were explicitly tested and confirmed to be effectively implemented by the target application.

CTRL-01: Filename Normalization, Extension Enforcement, and Path Traversal Resistance on Media Subsystem

Evaluated Target: POST /profile/image/file & GET /assets/public/images/uploads/{filename}

Standard Mapping: A01:2025 – Broken Access Control / A05:2025 – Security Misconfiguration (CWE-22 / CWE-624)
Test Methodology: Evaluated file upload processing (Content-Disposition header) and static asset retrieval by injecting single/double URL-encoded null-byte truncation sequences (cat.php%00.png), double extensions (cat.php.png), path traversal headers (../../cat.png), and relative traversal query paths (/assets/public/images/uploads/26.png/../../../../../server.js).

Observed Behavior: The backend neutralized all payload variants across both upload and retrieval handlers:

Upload Phase (POST): When testing for double extensions, null-byte truncation sequences, or path traversal headers, the backend consistently neutralized all payload variants. The application always uploaded and updated the image asset directly within the same designated target folder (/assets/public/images/uploads/), strictly maintaining the same server-assigned filename and .png extension (e.g., 26.png).

Retrieval Phase (GET): Requesting assets using relative path traversal sequences (/assets/public/images/uploads/26.png/../../../../../server.js) failed to break out of the webroot, safely collapsing via framework URL normalization without exposing restricted internal backend source code files (server.js).

CTRL-02: Restricted File Extension Whitelisting & Directory Traversal Protection on Static File Subsystem

Evaluated Target: GET /ftp/* (Restricted Document Repository)
Standard Mapping: A01/A05: Access Control & Misconfiguration (CWE-22, 200, 624)
Test Methodology: Evaluated static file delivery controls inside the /ftp/ repository against extension allowlist bypasses and directory traversal. Tested null-byte string truncation by appending allowlisted extension suffixes to target files (/ftp/package.json.bak%00.md, /ftp/package.json.bak%00.pdf) to trick the extension checker prior to file-read operations. Simultaneously evaluated relative directory breakout vectors using path traversal sequences from subdirectories (/ftp/quarantine/../../server.js).
Observed Behavior: The application strictly enforces an extension allowlist (permitting strictly public .md and .pdf documents). Null-byte injection payloads failed to bypass extension validation, returning blank/dropped responses without executing string truncation or serving the underlying target files (package.json.bak). Directory traversal attempts targeting restricted root files (server.js) were completely neutralized, while directory root breakouts (/ftp/quarantine/../../) safely defaulted to the public homepage (/) without exposing source code, backup artifacts, or internal directory listings.

CTRL-03: File Upload Size Limitation & Request Body Throttling

Evaluated Target: POST /profile/image/file (User Profile Avatar Upload)

Standard Mapping: A05:2025 – Security Misconfiguration (CWE-400: Uncontrolled Resource Consumption)

Test Methodology: Evaluated backend stream boundary controls and resource exhaustion resilience by padding a valid binary image payload with trailing dummy bytes, scaling the HTTP POST request body beyond 600 KB.

Observed Behavior: The Express/Multer file handling middleware actively monitored the incoming data stream and aborted processing at ~500 KB (524,288 bytes), returning an explicit MulterError: File too large error response. The backend enforces strict request body throttling and memory bounds, preventing Denial of Service (DoS) and disk space exhaustion via oversized media uploads.

CTRL-04: Stack Execution Isolation & Client-Side Image Context Binding

Evaluated Target: POST /profile/image/file & GET /assets/public/images/uploads/{filename}
Standard Mapping: A03:2025 – Injection / A05:2025 – Security Misconfiguration (CWE-79 / CWE-434)

Test Methodology: Evaluated media uploads for RCE and Stored XSS by submitting polyglot image payloads with embedded PHP code, raw HTML, and event handlers (<script>, onerror=alert(1)), analyzing backend interpreter execution capabilities and DOM rendering context.

Observed Behavior: The media pipeline neutralized RCE and Stored XSS vectors across two defensive layers:

Backend Stack Isolation (RCE Defense): Error probing confirmed the backend runs on Node.js/Express, lacking native PHP or CGI interpreters. Submitting scripts with arbitrary extensions (.php, .xml) cannot trigger server-side execution.

Client-Side Context Binding (Stored XSS Defense): Avatars render strictly via native HTML <img> elements (<img src="...">). Browsers enforce strict image-context parsing on <img> tags, treating inputs as binary data and ignoring embedded script markup—neutralizing Stored XSS regardless of Content-Type header handling.

SECTION 4: APPENDICES & FORMAL SIGN-OFF

Appendix A: CVSS v3.1 Severity & Remediation Priority Framework

Vulnerabilities are scored using CVSS v3.1 and mapped to OWASP Top 10:2025 categories. Each finding is assigned a Remediation Priority (P1–P5) establishing target resolution SLAs for developer triage.

SEVERITY	TARGET REMEDIATION SLA	CVSS v3.1 RANGE	DESCRIPTION & IMPACT
CRITICAL (P1)	**Immediate (24–48 Hours)**	9.0–10.0	Flaws permitting immediate, unauthenticated system compromise, full account takeover, or arbitrary code execution.
HIGH (P2)	**14 Days**	7.0–8.9	High-impact flaws enabling significant data exposure, privilege escalation, or full session hijacking.
MEDIUM (P3)	**30 Days**	4.0–6.9	Flaws requiring specific user interaction or specific environmental conditions to impact confidentiality, integrity, or availability.
LOW / INFO (P4)	**60 Days**	0.1–3.9	Minor security defects, configuration weaknesses, or defense-in-depth gaps with limited impact or high exploitation complexity.
INFORMATIONAL (P5)	**90 Days**	0.0	Non-vulnerable security best-practice recommendations, architectural hardening suggestions, or operational hygiene items.

Appendix B: Developer Remediation Action Plan Checklist

This checklist organizes technical mitigations by priority to guide engineering teams in systematically remediating identified risks:

PRIORITY	REMEDIATION ACTION ITEM
P1 - CRITICAL	**[FIND-01]** Enforce Parameterized SQL Queries & Prepared Statements; Implement Strict Server-Side Input Validation & Encoding; Enforce Principle of Least Privilege for Database Accounts
P3 - MEDIUM	**[FIND-07]** Enforce Server-Side Object-Level Authorization (BOLA Mitigation); Implement Cryptographic or Non-Sequential Resource Identifiers; Ensure Client-Server State Consistency
P3 - MEDIUM	**[FIND-02]** Eliminate Credential Over-Serialization in API Responses; Migrate to Strong Password Hashing Algorithms (Argon2id / bcrypt); Store Session Tokens in Secure HttpOnly Cookies
P3 - MEDIUM	**[FIND-04]** Relocate Confidential Files & Restrict Directory Access; Implement Global Graceful Exception Handling; Enforce Strict API Schema & Data Type Validation
P3 - MEDIUM	**[FIND-05]** Enforce Strict IP & Domain Destination Allowlists; Block Loopback, RFC 1918 Private Subnets & Cloud IMDS; Implement Egress Network Segmentation & Firewall Restrictions
P3 - MEDIUM	**[FIND-06]** Bind Author Identity Strictly to Verified Session Tokens; Enforce Server-Side Object-Level Access Control; Audit Subsystem Non-Repudiation Controls
P4 - LOW	**[FIND-03]** Server-Side Image Re-Encoding & EXIF Stripping; Isolate Static Upload Storage Outside Webroot; Enforce Access Controls on Uploaded Assets

Appendix C: Assessment Environment & Testing Tooling

Testing was conducted in a local staging environment using **Burp Suite Community Edition (v2025.3.4)** for live interception, parameter tampering, and manual payload replaying via Repeater and Intruder, routed via **FoxyProxy** extension. Endpoint discovery and static directory fuzzing were executed using **ffuf**, while ProjectDiscovery **Interactsh** handled out-of-band callback verification. Client-side auditing relied on **Firefox Developer Tools** (Console tab for document.cookie and HttpOnly flag evaluations; Storage tab for LocalStorage inspection) alongside **GhostJS** burp extension to scan frontend HTML tags and scripts for interesting endpoints (frontend and backend). All test vectors—including manual binary byte-padding in Repeater for upload stream boundary probing (~500 KB Multer limit), path traversal (../../server.js), null-byte truncation (%00, %2500), and more—were manually engineered using non-harmful custom security payloads.

Appendix D: Artifact Cleanup Attestation & Formal Sign-Off

Artifact Cleanup Attestation: The Principal Security Assessor formally attests that all temporary test accounts, elevated admin session tokens, uploaded test file payloads in /uploads/, spoofed product review entries, tampered cart items, and proxy history logs generated during the active testing window (October 5-19, 2026) were safely purged from the target deployment upon engagement completion.

Legal Disclaimer & Confidentiality Notice

This document is a technical security evaluation report produced by Verge Security Labs solely for demonstration and portfolio evaluation purposes. Technical findings reflect security evaluations conducted against a controlled, open-source target environment (OWASP Juice Shop v20.2.0). To maintain realistic technical fidelity, HTTP request schemas, host paths, and target endpoints are preserved in complete operational detail, while authentication tokens, session headers, and user credentials have been scrubbed and sanitized. This deliverable contains no proprietary corporate assets. All text, layout architecture, and report intellectual property herein remain the exclusive property of Verge Security Labs. Unauthorized copying, re-branding, plagiarized redistribution, or commercial reproduction of this report without explicit written consent is strictly prohibited.

Lead Assessor Formal Sign-Off

NAME / ENTITY	TITLE / ROLE	ORGANIZATION	STATUS
Alex Vance	Principal Security Assessor	Verge Security Labs	DIGITALLY SIGNED & VERIFIED